



Especificación: DSL
Análisis, Diseño e Implementación

CodeLAB: Proyecto colaborativo

DSL / Motor de consultas de datos relacionales

Handlebars para generar HTML dinámico

Sass/SCSS: generador de Landing Pages

1.a. Enunciado de Caso de Uso (Use Case Specification)

Nombre del Caso de Uso: ID: UC-001

Actor Principal:

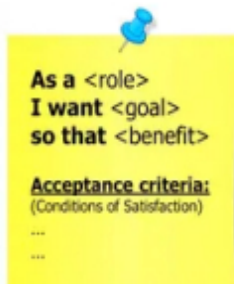
Precondiciones:

-
-

Postcondiciones:

-
-

1.b. User Story (Historia de Usuario)



Título:

AS a:

...

I want:

...

So that:

....

Criterios de Aceptación:

-
-
-

2. Criterios de Diseño.

2.a. Arquitectura WEB:

-
-



Tecnología WEB & "Domain-Specific Languages" (DSLs)

Problemática.

Tecnología Web. En una estructura de archivos de un sitio WEB:

¿Cual de las tecnologías que convergen en los Websites, se considera formalmente un DSL?

En la estructura de archivos que incluyen HTML, CSS, JavaScript, JSON ninguno de ellos se considera formalmente un DSL (Domain-Specific Language) en el sentido estricto de la informática.

A continuación la explicación. del porqué en cada caso:

a. Análisis por archivo (index.html)

HTML5 (HyperText Markup Language)

No es un DSL

Sustentación: Aunque se usa para un dominio específico (documentos web), el consenso técnico actual es que HTML es un Lenguaje de Marcación (Markup Language), no un lenguaje de programación. No tiene lógica de control (if/else), variables ni funciones. Es un DSL declarativo en un sentido muy amplio, pero técnicamente se clasifica como un lenguaje de marcado.

b. Análisis por archivo (styles.css)

CSS (Cascading Style Sheets)

No es un DSL

Sustentación: CSS es un Lenguaje de Hoja de Estilos. Es declarativo y específico del dominio de la presentación visual. Sin embargo, no tiene capacidad de programación (no puedes hacer bucles ni lógica compleja nativamente en CSS puro). Se considera un DSL declarativo para el dominio de estilos, pero no un lenguaje de programación.

c. Análisis por archivo (script.js)

JavaScript

No es un DSL

Sustentación: JavaScript es un Lenguaje de Propósito General (GPL). Es Turing-completo. Se puede usar para casi cualquier tarea (web, servidores, móviles, juegos). No está restringido a un dominio específico como "bases de datos" o "sensores".

c. Análisis por archivo (datos.json)

JSON (JavaScript Object Notation)

No es un DSL

Sustentación: JSON es un formato de intercambio de datos, no un lenguaje de programación. No tiene sintaxis de control, lógica ni comandos. Es un formato de datos estructurado (Data Interchange Format), usado para transportar información, pero no para "programar" o "ejecutar" lógica de dominio.



Domain-Specific Languages (DSL)

¿Por qué no son DSLs formales?

Un DSL (Domain-Specific Language) formalmente es un lenguaje de programación diseñado para resolver problemas en un dominio específico y que permite expresar soluciones en ese dominio de manera más natural y concisa que un lenguaje de propósito general.

Características clave que definen un DSL:

- Dominio Específico: Resuelve problemas de un área concreta (ej. SQL para bases de datos, AST - parser).
- Capacidad de Expresión: Permite definir lógica, reglas, transformaciones o comportamientos dentro de ese dominio.
- Abstracción: Oculta los detalles complejos de la implementación subyacente.

En el entorno de la tecnología WEB:

HTML/CSS: Son lenguajes de marcado y estilos, respectivamente. Son declarativos y específicos del dominio web, pero carecen de la capacidad de programación lógica (aunque CSS moderno tiene algunas capacidades limitadas como @media, no es lo suficientemente potente para considerarse un lenguaje de programación completo).

A menudo se les llama "pseudo-DSLs" o "lenguajes de descripción", pero no DSLs de programación.

Ejemplos reales de DSLs que están embebidos en la tecnología Web.

- SQL: Para hacer query a bases de datos (DSL para datos relacionales).
- Verilog/VHDL: Para diseñar circuitos electrónicos (DSL para hardware).
- LaTeX: Para componer documentos científicos con fórmulas matemáticas (DSL para edición de texto académico).
- Gradle/Maven: Para definir la construcción de software (DSL para compilación).
- Regular Expressions (Regex): Para buscar patrones en texto (DSL para texto).

Conclusión: En tu proyecto web, no estás utilizando un DSL. Estás utilizando un conjunto de tecnologías estándar para la web (HTML, CSS, JS, JSON). Si quisieras implementar un DSL en este contexto, tendrías que crear un lenguaje nuevo o usar uno existente para algo específico, por Ejemplo:

- Usar SQL dentro de tu script.js para hacer queries a una base de datos (SQL sería el DSL).
- Usar Sass/SCSS (un preprocesador CSS que es un DSL sobre CSS) en lugar de CSS puro.
- Usar Jinja2 o Handlebars (plantillas que son DSLs para generar HTML dinámico) en lugar de inyectar HTML directamente con JS.



Proyectos Colaborativos (Tres participantes).

Proyectos sugeridos por cada caso de uso, diseñados para ser presentados como proyectos de fin de semestre en Ingeniería de Software. Cada proyecto demuestra claramente la implementación de un DSL específico en un contexto web.

1. Usar SQL dentro de tu script.js para hacer queries a una base de datos

1.a. Proyecto A: Sistema de Gestión de Inventario con Consultas SQL Dinámicas

Descripción: Crear una aplicación web completa de gestión de inventario donde los usuarios puedan realizar consultas complejas mediante una interfaz que genera y ejecuta consultas SQL directamente desde JavaScript.

Características clave:

Interfaz web con formulario para construir consultas SQL de forma visual

Validación de consultas SQL antes de ejecución

Dashboard con gráficos generados desde resultados de SQL

Sistema de permisos basado en roles SQL

Exportación de reportes en múltiples formatos

Tecnologías: Node.js, SQLite/PostgreSQL, Express, SQL.js o librería de base de datos, React/Vue

1.b. Proyecto B: Analítica de Ventas con Motor de Consultas SQL

Descripción: Plataforma de análisis de ventas que permite a usuarios no técnicos crear consultas SQL personalizadas para extraer insights de datos de ventas.

Características clave:

Editor SQL con autocompletado y validación en tiempo real

Predefinición de consultas SQL comunes (templates)

Sistema de alertas basado en condiciones SQL

Visualización interactiva de resultados de queries

Historial y versionado de consultas SQL guardadas

Tecnologías: PostgreSQL, Node.js, D3.js para visualización, SQL parser library

2. Usar Sass/SCSS en lugar de CSS puro

2.a. Proyecto A: Sistema de Diseño Modular con Variables SCSS

Descripción: Desarrollar un sistema de diseño completo (design system) utilizando SCSS que permita a equipos crear interfaces consistentes con theming dinámico.

Características clave:

Biblioteca de componentes reutilizables con SCSS mixins

Sistema de variables SCSS para colores, tipografía, espaciado

Generación automática de temas (claro/oscuro/personalizados)

Documentation site con ejemplos interactivos

Build pipeline que compila múltiples temas desde un solo código SCSS

Tecnologías: Sass/SCSS, Gulp/Webpack, Storybook, CSS Variables